

Scanner Server Migration — IT Handoff

For: IT (server / infrastructure team) **From:** Dan Plant, Technical Superintendent, Well Control Engineering **Date:** 2026-08-27

Goal: move the dashboard scanner off Dan's workstation onto the sacred server (sdr1azneu1s01d.corp.local) so the dashboards update 24/7, independent of Dan's PC being on.

1. What runs today (current state, all on Dan's workstation)

Everything lives in one folder: C:\TSC-Dashboard.

- scripts\Update-Dashboard.ps1 ("the scanner", currently v2.32) runs as a **Windows Scheduled Task every 10 minutes** under Dan's account, in **Windows PowerShell 5.1** (deliberately 5.1-compatible — no modules, no internet access, no PowerShell 7 required).
- **Inputs:** two OneDrive-synced SharePoint folders under Dan's profile — "Technical Services DMS - Fleet ERTs and BWM Reporting" → **TSC REPORTING** and **PLANNING REPORTING**. Rig crews' report tools post .json files (and weekly .xlsx planning workbooks) into these SharePoint libraries via an existing Power Automate flow; OneDrive syncs them to Dan's PC; the scanner reads the local synced copies. **The files ARE the database** — there is no other data store.
- **Outputs:** the scanner writes data files locally, then copies them to the IIS content shares on the sacred server: \\sdr1azneu1s01d\sacred\dashboard (reports data + per-report copies), \\sdr1azneu1s01d\sacred (BOP planning data, SSCE requests data), and \\sdr1azneu1s01d\SSORT (the SSCE→COC write-back review copy).
- **State files** in the folder root remember what has already been announced so nothing double-fires: notified-state.json, break-ins-notified-state.json, break-ins-pending.json, ssce-notified-state.json, ssce-notifications-pending.json. These **MUST** move with the installation.
- **Config:** config.json in the folder root holds every path (reportFolders, deployPath, bopDeployPath, cocDashboardPath, notification settings). No paths are hardcoded in the script.

A full scan takes roughly 1-5 minutes; the slowest steps are the COC write-back (parses a large HTML file) and the network copies — both get faster on the server (§3, step 3).

2. The ONE design decision for the meeting: report source access

On the server there is no interactive OneDrive client signed in as Dan. The scanner just needs a **local folder path containing the current contents of the two SharePoint libraries** — how that folder is kept current is IT's choice:

Option	How	Assessment
A. Scheduled SharePoint pull (recommended)	A small pre-step (PnP.PowerShell or Graph API with an app registration, read-only access to the two libraries) syncs the libraries to e.g. D:\TSC-Dashboard\incoming\ before each scan, or on its own 5-min schedule	Clean, service-account friendly, no interactive login. Read-only permission is sufficient — the scanner never writes to the source
B. OneDrive sync under a service account	Sign a service account into OneDrive on the server and sync the libraries	Works but Microsoft discourages the OneDrive sync client on servers; interactive login/token refresh is a maintenance liability

Option	How	Assessment
C. WebDAV / mapped drive to SharePoint	Map the library as a network path	Fragile (auth timeouts); not recommended

Whichever option is chosen, the result is just a path (or two) that goes into `config.json` → `reportFolders`. Nothing in the scanner changes.

Deletions matter: occasionally bad files are deleted at source (this happened this week). The pull mechanism should mirror deletions (remove local files no longer in the library), or bad files will keep being scanned forever.

3. Migration steps

1. **Copy the whole** `C:\TSC-Dashboard` **folder** from Dan's PC to the server (suggested: `D:\TSC-Dashboard`). This brings the script, the dashboards, the config, and — critically — the state files (§1).
2. **Unblock the files** so the script runs without the security prompt: `Get-ChildItem D:\TSC-Dashboard -Recurse | Unblock-File`
3. **Edit** `config.json`: `reportFolders` → the server-local synced folder(s) from §2. `deployPath` / `bopDeployPath` → since the IIS shares live on this very server, point these at the **local** filesystem folders behind the “sacred” share instead of the UNC paths (faster, no network dependency; UNC also still works). `cocDashboardPath` → the local folder behind the SSORT share. `notifications` → see §4 before enabling.
4. **Register the Scheduled Task.** Action: `powershell.exe -NoProfile -ExecutionPolicy Bypass -File D:\TSC-Dashboard\scripts\Update-Dashboard.ps1` · Start in: `D:\TSC-Dashboard\scripts` · Trigger: every 10 minutes, indefinitely · Setting: **“Do not start a new instance”** if the previous run is still going · Run as: a **service account / gMSA** with (a) read on the report source folder, (b) write on the IIS content folders and the SSORT folder, (c) “Run whether user is logged on or not”. Do NOT run it as Dan's personal account.
5. **First run by hand** (as the service account) and compare its console output against a run on Dan's PC the same hour — report counts and warnings should match (a couple of known warnings about skipped daily-checks files with no rig identity are expected and correct).

4. Notifications — one thing that will NOT survive the move as-is

`config.json` currently has `"notifications": { "method": "outlook", ... }` — that sends email through the **locally signed-in Outlook on Dan's PC**. There is no Outlook on the server, so after migration either:

- switch to `"method": "smtp"` with `smtpServer` set to the internal relay + a shared sending mailbox — this is the same SMTP relay request Dan already has open with IT; this meeting is a good time to close it — or
- set `"enabled": false` for now. The bigger, position-based notification system is being built separately in Power Automate (see NOTIFICATION-DISTRIBUTION-IT-HANDOFF.pdf) — once that flow is live this scanner-side notifier stays off anyway, to avoid double emails.

5. Acceptance checklist

1. Manual run on the server completes green: report count matches Dan's PC, same (expected) warnings, all “Deployed ...” lines print.
2. Dashboards load in a browser and show current data (check the last-updated stamps on each page).

3. Post a test report through a report tool → it appears on the dashboard within ~10 minutes without anyone touching anything.
4. Delete the test file at source → it disappears from the local sync on the server (confirms deletion mirroring, §2).
5. State files in D:\TSC-Dashboard show fresh timestamps after runs.
6. **Only then:** disable (don't delete) the scheduled task on Dan's PC. Two scanners running in parallel is safe short-term (same inputs, same outputs, newest-wins) but pointless long-term.

6. Rollback

Re-enable the scheduled task on Dan's PC and disable the server one. The installation there is untouched by this migration. Nothing else to undo.

7. Ongoing maintenance

- **Script updates:** Dan receives updated Update-Dashboard.ps1 versions periodically. Give Dan (or agree a drop process for) write access to D:\TSC-Dashboard\scripts so updates don't need a ticket; each update is one file replaced + Unblock-File.
- **Version check:** the script prints its version on every run ("TSC Dashboard scanner v2.xx"); the scheduled task history shows it.
- **Monitoring (optional but cheap):** alert if the task's last result is non-zero or if reports-data.js in the IIS folder is older than 30 minutes.

Technical reference: HANDOFF.md and INTEGRATION-CONTRACT.md in the project repo document the full pipeline; Update-Dashboard.ps1 is self-documenting. Questions on content/behaviour → Dan Plant.